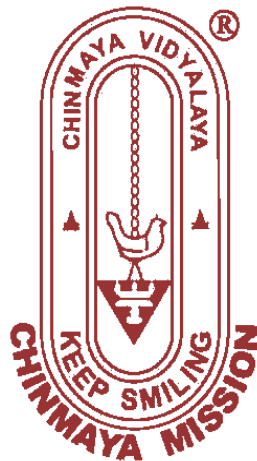


CHINMAYA VIDYALAYA ANNANAGAR

COMPUTER SCIENCE

FOOD ORDERING AND MANAGEMENT SYSTEM WITH SMART USER INTERFACES AND ORDER TRACKING 2024-2025



**A Project Report Submitted
By
T. NIVEDHA
Class: XII B**

EXAM ROLL NO: _____

CHINMAYA VIDYALAYA, ANNANAGAR
*Plot No. 5063A, Z-Block, Belly Area,
Anna Nagar, Chennai-600 040*

BONAFIDE CERTIFICATE

This is to certify that T. NIVEDHA of class XII B has performed the project titled FOOD ORDERING AND MANAGEMENT SYSTEM WITH SMART USER INTERFACES AND ORDER TRACKING at CHINMAYA VIDYALAYA, Anna Nagar, during the academic year 2024-25.

Submitted for Practical Examination held on _____ at CHINMAYA VIDYALAYA, Anna Nagar.

Teacher in-charge:

Signature of the Principal

Signature of External Examiner

ACKNOWLEDGEMENT

I sincerely thank our principal **Mrs. V. Gowrilakshmi** for allowing me to do this project.

I am deeply grateful to our Computer Science teacher, **Mrs. V. Gnanavadiu** for her unwavering support and expert guidance. Her encouragement and insights were invaluable in shaping this project.

Finally, I would also like to thank my parents who helped me a lot in completing this project within the deadline.

INDEX

S.NO	TOPIC
1.	INTRODUCTION
2.	SYSTEM DESIGN
3.	TESTING AND RESULTS
4.	SOURCE CODE
5.	CONCLUSION AND FUTURE WORK
6.	BIBLIOGRAPHY

1. INTRODUCTION

Python is a widely used general-purpose, high-level programming language. It was initially designed by Guido van Rossum in 1991 and developed by Python Software Foundation. It was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code. Python's extensive standard library and rich ecosystem of third-party modules empower to build a wide range of applications, from web development and data science to artificial intelligence and machine learning.

Apart from Python and SQL, which are part of the prescribed textbook, Tkinter was recommended for study and is covered in the latter part of the book. In addition to these, the following are the reasons for selecting Python, SQL, and Tkinter for this project

Python:

- Python is a free open-source software
- Python can be used on many platforms like Linux, Windows and Macintosh. So it is platform-independent.
- Python's syntax is clean and easy to learn, making it ideal for rapid development.
- Python emphasizes code readability, shorter codes, ease of writing and closer to the common English language.
- Python's extensive standard library provides modules for various tasks, reducing development time.
- Python is suitable for a wide range of applications, including web development, data science, and machine learning.
- A large and active community provides extensive documentation, tutorials, and libraries.

SQL:

- SQL is a powerful language for managing relational databases, essential for storing and retrieving information about customers, orders, menu items, and inventory.
- SQL ensures data consistency and accuracy through features like constraints, triggers, and stored procedures.
- SQL can handle complex queries and data manipulations, allowing for efficient data analysis and reporting.

Tkinter:

- Tkinter is a simple and straightforward library for creating Graphical User Interface (GUI) applications.
- Tkinter applications can run on various operating systems without significant modifications.
- Tkinter allows for quick development of user interfaces without complex setups.
- Tkinter is seamlessly integrated with Python, making it easy to create interactive and user-friendly applications.

By combining these technologies, the project leverages its strengths to create a robust and efficient food ordering and management system. Python's versatility and readability, SQL's data management capabilities, and Tkinter's user-friendly interface development make it a suitable choice for this project.

The Python program developed in this project interacts with the MySQL database to manage a food ordering system. It uses Tkinter for creating a graphical user interface (GUI) on a desktop application. Since Tkinter user interfaces are not intended to develop a web application, both the customer and Restaurant modules are kept in the same system along with the SQL Server. However, for a small-scale operation, a desktop application using Tkinter can

be a cost-effective and efficient solution, especially if the focus is on internal management rather than external customer interaction.

In this project, smart user interfaces (panels) are created using Tkinter. Here, smart means the same panel will adopt changes without change in the code. For example, the panel used by the customer for Customer Registration will change to the Restaurant Manager when he wants to view the registration details. But these two panels are adoptively displayed and handled by the same function `Customer_Register_ShowDetails()` developed in this project.

Customers can track the status of their orders, from placement to delivery. The system helps restaurant staff manage orders efficiently and reduce wait times. Overall, the main objectives of the system are to:

- i) Develop a software system that allows customers to order food items from a restaurant and assists restaurant staff in managing these orders.
- ii) Create smart user interfaces or panels to enable efficient use of the system.
- iii) Improve Efficiency: Streamline order-taking, food preparation, and delivery processes.
- iv) Reduce Operational Costs: Optimize inventory management and assign persons effectively to deliver the food.

By incorporating these features, the system strives to provide a seamless and satisfying dining experience for customers while simplifying operations for restaurant staff.

2. SYSTEM DESIGN

The system is designed as a user-friendly application for managing orders in a restaurant. It offers separate interfaces for customers and restaurant management, each with distinct functionalities accessible via panels in a GUI. The application's GUI is developed using Tkinter and organized into separate windows (or "panels") for better modularity and user experience. Each function in the GUI is equipped with labels, input fields, and buttons to guide user interactions. Additionally, message box alerts handle error messages or confirmations, enhancing user feedback.

2.1 Overall System Design

The overall system design is shown in Figure 1 along with the function names. Each function is responsible for creating a specific panel or window, making the code modular and easier to understand. By identifying the function names relevant code for each panel can be easily located in the program given in this project report.

1. Main Window

This serves as the primary navigation point, allowing users to select either the Restaurant or the Customer Panel. Customers have the access for registerig, viewing orders and placing new orders, while restaurant staff can access administrative functions.

2. Customer Register / View Panel

This panel helps the customer to choose either the Customer Registration option, the Place Order option, or the View Accepted Order option. Selecting one of these options opens the corresponding panels: CUSTOMER REGISTRATION DETAILS, PLACE ORDER, or ORDER ACCEPTED panel respectively as shown in Figure 1.

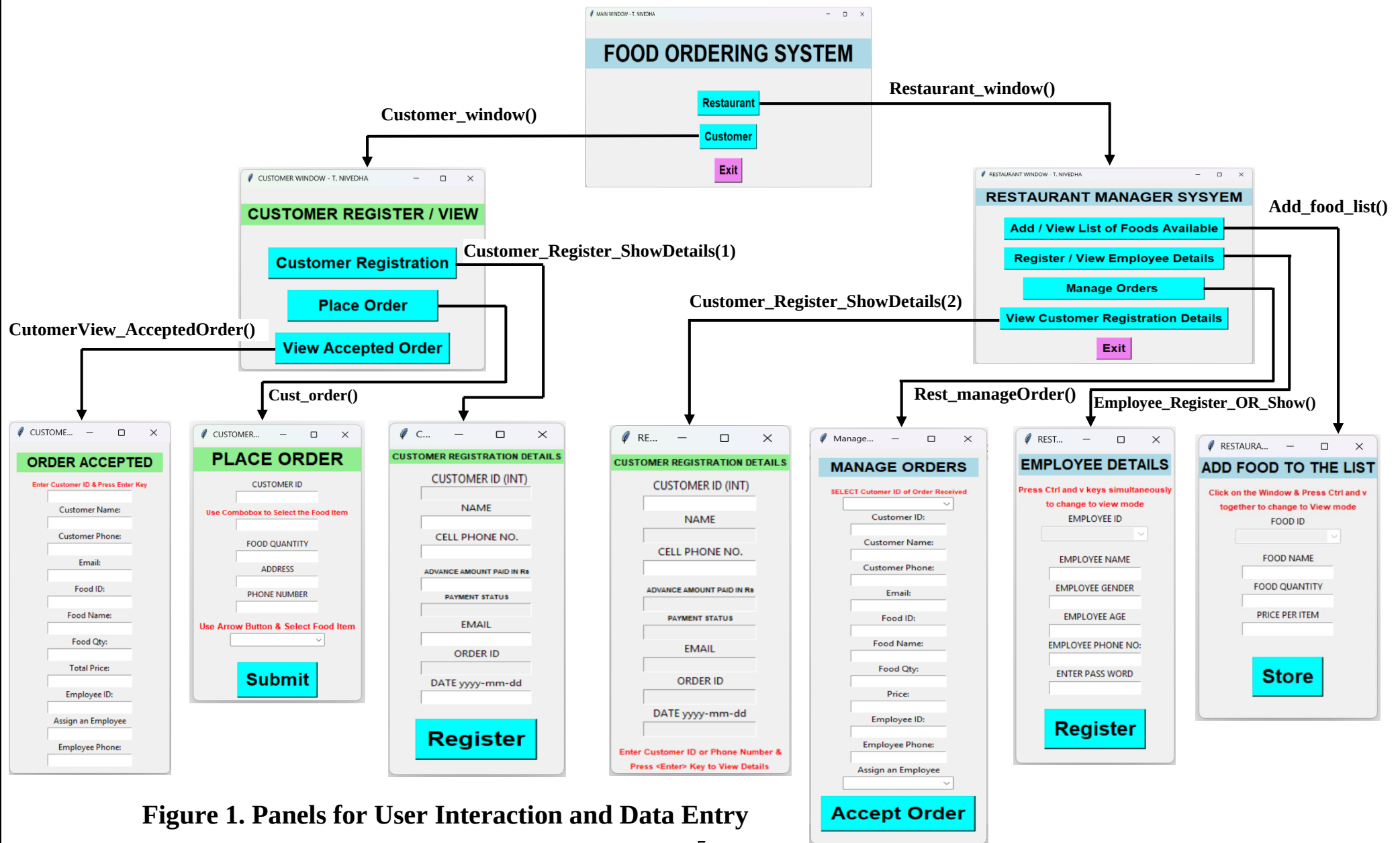


Figure 1. Panels for User Interaction and Data Entry

- **CUSTOMER REGISTRATION DETAILS:** This panel allows new customers to register by entering their details into the system. A customer ID will be provided to each customer after registration. The customers will be required to enter their correct mobile number because it will be used as a security check when they place an order.
- **PLACE ORDER:** This panel enables customers to place new food orders by selecting items and quantities. They have to use their customer ID to order the food. In addition to this customer ID, the order placed will be cross-checked with the mobile number already registered.
- **ORDER ACCEPTED:** This panel provides customers with an option to view the status and details of accepted orders by entering their unique Customer ID. This view shows customer information, order details (such as food items and quantity), and assigned employee information for delivering the food.

3. Restaurant Panel

Selecting the Restaurant option in the main window opens the Restaurant Manager System panel. The four options Add / View List of Foods Available, Register / View Employee Details, Manage Orders, and View Customer Registration Details open the corresponding panels: ADD FOOD TO THE LIST, EMPLOYEE DETAILS, MANAGE ORDERS, AND CUSTOMER REGISTRATION DETAILS respectively as shown in Figure 1.

- **ADD FOOD TO THE LIST:** This panel allows restaurant staff or managers to add available food items for delivery. By pressing the CTRL and 'v' keys together, the panel changes as shown in *Figure 2(a)*, allowing the manager to view food item details (view mode), such as price, at a later time. So this is named a **Smart Panel** or

Smart User Interface and no separate function or code is written to display this view panel.

The figure shows three distinct user interface panels for a restaurant management system. Panel (a) is titled 'ADD FOOD TO THE LIST' and includes input fields for 'FOOD ID', 'FOOD NAME', 'FOOD QUANTITY', and 'PRICE PER ITEM', along with a red 'Store' button. Panel (b) is titled 'EMPLOYEE DETAILS' and contains fields for 'EMPLOYEE ID', 'EMPLOYEE NAME', 'EMPLOYEE GENDER', 'EMPLOYEE AGE', 'EMPLOYEE PHONE NO:', and 'ENTER PASS WORD', with a red 'Register' button. Panel (c) is titled 'CUSTOMER REGISTRATION DETAILS' and features fields for 'CUTOMER ID (INT)', 'NAME', 'CELL PHONE NO.', 'ADVANCE AMOUNT PAID IN R\$', 'PAYMENT STATUS', 'EMAIL', 'ORDER ID', and 'DATE yyyy-mm-dd', with a red 'Register' button at the bottom.

(a) Food List Panel
(View Mode)

(b) Employee Panel
(View Mode)

(c) Customer Registration
(Search Mode)

Figure 2. Smart User Interfaces

- **EMPLOYEE DETAILS:** The Restaurant Manager can register an employee who wants to deliver food to the customers. The Manager can also view the employee's details later when he wants the employee's phone number or other details. The same panel used for registration is **smartly adapted** to allow selecting a specific employee and viewing their details, with clear instructions displayed on the panel to switch to this view mode as shown in *Figure 2(b)*.
- **MANAGE ORDERS:** Provides an interface for managing customer orders, viewing order statuses, and assigning them to available employees.
- **CUSTOMER REGISTRATION DETAILS:** This feature assists the restaurant manager in effectively tracking and managing customer data. If a customer forgets the customer ID, they can

contact the restaurant manager and provide their registered phone number. The manager can then use this phone number to retrieve the customer ID associated with that phone number. The same panel used by customers for registration is **smartly adapted** to allow searching and displaying customer details by either customer ID or phone number. In both cases, the panel dynamically adjusts or is modified as shown in *Figure 2(c)* by calling a single function (**Customer_Register_ShowDetails()**) to perform the required action.

2.2 Database Design

The database is a crucial component of the system, storing essential information for smooth operations. Figure 3 shows the tables created in this project to store information in the SQL database.

Field	Type	Null	Key	Default	Extra
c_id	int	YES		NULL	
c_name	varchar(255)	NO		NULL	
cphone	bigint	YES		NULL	
payment	int	YES		NULL	
pstatus	varchar(20)	YES		NULL	
email	varchar(255)	YES		NULL	
orderid	int	YES		NULL	
date	date	YES		NULL	

3(a) Customer table (for customer registration.)

Field	Type	Null	Key	Default	Extra
Emp_id	int	YES		NULL	
ename	varchar(255)	NO		NULL	
emp_g	varchar(10)	YES		NULL	
eage	int	YES		NULL	
emp_phone	bigint	YES		NULL	
pwd	varchar(255)	YES		NULL	

3(b) Employee table (for employee registration.)

Field	Type	Null	Key	Default	Extra
Food_id	int	NO	PRI	NULL	auto_increment
Food_name	varchar(255)	YES		NULL	
Food_quantity	int	YES		NULL	
Price_per_item	int	YES		NULL	

3(c) Food_list table (for adding food details.)

Field	Type	Null	Key	Default	Extra
c_id	int	YES		NULL	
Food_name	varchar(255)	YES		NULL	
Food_qty	int	YES		NULL	
Address	varchar(255)	YES		NULL	
cphone	bigint	YES		NULL	

3(d) Cust_order table (for storing customer orders.)

Field	Type	Null	Key	Default	Extra
orderid	int	NO	PRI	NULL	auto_increment
c_id	int	YES		NULL	
c_name	varchar(255)	NO		NULL	
cphone	bigint	YES		NULL	
email	varchar(255)	YES		NULL	
Food_id	int	YES		NULL	
Food_name	varchar(255)	YES		NULL	
Food_qty	int	YES		NULL	
Price	int	YES		NULL	
Emp_id	int	YES		NULL	
ename	varchar(255)	YES		NULL	
emp_phone	bigint	YES		NULL	

3(e) Rest_manageorder table (for assigning employee and accepting orders.)

Field	Type	Null	Key	Default	Extra
Order_index	int	NO	PRI	NULL	auto_increment
Ordered_cust_ids	int	YES		NULL	

3(f) Customer_orders table

(for storing IDs of customers who have placed orders.)

Figure 3. Tables Created Using SQL Database

The detailed breakdown in Table 1 provides a comprehensive overview of the code or functions of the system's architecture and its interaction with the underlying database.

Function	Subfunctions	Event Functions	MySQL Tables Used	Description
Main_window	-	-	-	Allows users to choose between the Customer Panel and Restaurant Panel .
Customer_window	-	-	-	Provides options for customer-related actions such as registration, placing orders, and viewing accepted orders.
Customer_Register_ShowDetails	Register_cust_details	-	Customer	Registers a new customer by storing customer details like name, phone number, and email.
	Show_cust_details	-	Customer	Retrieves and displays customer details based on customer ID or phone number.
	check_entry_c1_and_process	c1.bind('<Return>', check_entry_c1_and_process)	Customer, Rest_manageOrder	Checks if a customer ID exists and retrieves order details if available. Displays an error if no matching record is found.
	check_entry_c3_and_process	c3.bind('<Return>', check_entry_c3_and_process)	Customer, Rest_manageOrder	Verifies if a phone number exists in the database and retrieves order details if matched.
Cust_order	populate_combo	food_combo.bind("<< ComboboxSelected>>", update_oe2)	Food_list	Populates the food item selection combo box with available food names from the Food_list table.
	update_oe2	-	-	Updates the selected food item entry field when a food item is chosen from the combo box.
	Submit_food_order	-	Customer, Cust_order, Customer_orders	Validates customer details and submits a new food order, recording order details and the associated customer ID.
	Show_food_order	-	Cust_order	Displays details of a customer's existing food order, including food items and quantities.
CustomerView_AcceptedOrder	get_c_id_entry	c_id_entry.bind("<Return>", get_c_id_entry)	Rest_manageOrder	Retrieves and displays accepted order details based on customer ID, including order and assigned employee details.
Restaurant_window	-	-	-	Provides options for restaurant-related actions

Add_food_list	Store_food_details	-	Food_list	Adds new food items to the list, specifying food name, quantity, and price per item.
	Show_food_details	food_id_combo.bind("<<ComboboxSelected>>", on_food_id_selected)	Food_list	Displays available food items, allowing staff to view and update inventory details.
Employee_Register_OR_Show	Register_emp_details	-	Employee	Registers a new employee by storing details like employee name, ID, and phone number.
	Show_emp_details	e1_combo.bind("<<ComboboxSelected>>", on_Emp_id_selected)	Employee	Retrieves and displays employee details based on employee ID.
Rest_manageOrder	populate_combo_employee	-	Employee	Populates the employee selection combo box with available employee names for assigning orders.
	select_employee	assign_emp_combo.bind("<<ComboboxSelected>>", select_employee)	Employee	Retrieves and displays details of a selected employee, including ID and phone number.
	poulate_orderCombo	-	Customer_orders	Populates the order selection combo box with IDs of received customer orders.
	assign_selected_value	orders_combo.bind("<<ComboboxSelected>>", lambda event: assign_selected_value(orders_combo.get()))	Customer, Cust_order	Retrieves selected customer and order details for processing in the Manage Orders interface.
	select_order_andAccept	-	Customer, Cust_order, Food_list	Retrieves and calculates order details, including total price based on food quantity and price per item.
	Accept_order	-	Rest_manageOrder	Confirms and records the accepted order in the database, including assigned employee details.

Table -1 Functions, the corresponding Subfunctions, Event functions, and the Corresponding MySQLTables

3. TESTING AND RESULTS

The following hardware and software were used to test the software system developed in this project.

Hardware:

Processor-intel core i5-2410M Processor 2.30 GHz

RAM- 8 Gb

Software:

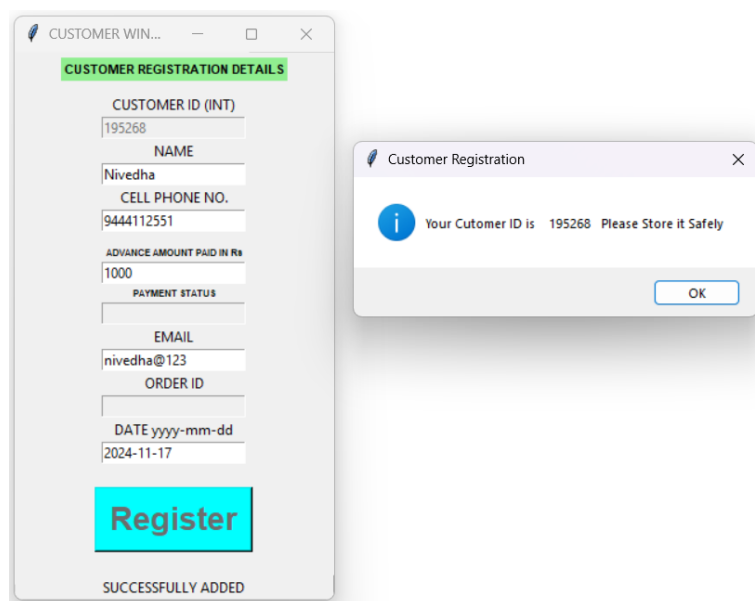
Operating system- windows 8

Language- Python 3.10

Database- MySQL 8.3

MS Word- For drawing the diagrams and preparing the project report

The screenshots of the panels created in this project are already presented in *Figure 1*. Figure 4 shows the customer registration panel along with the message “SUCCESSFULLY ADDED” when the registration is successfully done. It also instructs the customer to store their customer ID safely by displaying the ID in a message box. Figure 5 shows the records entered by using this panel.



The image displays two screenshots from a software application. The primary screenshot is a 'CUSTOMER REGISTRATION DETAILS' form window. It contains several input fields: 'CUSTOMER ID (INT)' with the value '195268', 'NAME' with 'Nivedha', 'CELL PHONE NO.' with '9444112551', 'ADVANCE AMOUNT PAID IN Rs' with '1000', 'PAYMENT STATUS' (empty), 'EMAIL' with 'nivedha@123', 'ORDER ID' (empty), and 'DATE yyyy-mm-dd' with '2024-11-17'. A prominent blue 'Register' button is at the bottom. Below the button, the text 'SUCCESSFULLY ADDED' is visible. The second screenshot is a smaller 'Customer Registration' message box. It features an information icon and the text: 'Your Customer ID is 195268 Please Store it Safely'. An 'OK' button is located at the bottom right of this message box.

Figure 4. Customer Registration


```
mysql> SELECT * FROM Customer;
```

c_id	c_name	cphone	payment	pstatus	email	orderid	date
195268	Nivedha	9444112551	1000		nivedha@123	NULL	2024-11-17
222170	Anbu	9444112579	1000		anbu@yahoo.com	NULL	2024-11-18
947659	Raja	9884712551	2000		kkt@gmail.com	NULL	2024-11-18

Figure 5. Customer Table and Sample Records

Figure 6 shows the panel to be used to add food details. This is to be done by the restaurant manager such that the customer will know the food available in the restaurant and its price. The records of food details entered by using this panel are shown in Figure 7.

(a) Adding Food Details

(b) Viewing Food Details

Figure 6. Adding and Viewing Food Details

```
mysql> SELECT * FROM FOOD_LIST;
```

Food_id	Food_name	Food_quantity	Price_per_item
1	IDLY	1	25
2	DOSA	1	100
3	PONGAL	1	60
4	VADA	1	20

Figure 7. Food_list Table and Sample Records

The details of employees available to deliver food are to be entered and registered by the restaurant manager by using the panel shown in Figure 8(a). The manager can use the same panel to view all the details of any employee later by just pressing choosing CTRL and ‘v’ keys together. This will enable the combo box in that panel and he can choose the employee ID from that to view the corresponding details as shown in Figure 8(b). The employee details entered are shown in Figure 9.



Figure 8. Employee Details

```
mysql> SELECT * FROM Employee;
```

Emp_id	ename	emp_g	eage	emp_phone	pwd
158732	BABU	M	25	9444112556	babu@321
493422	RAVI	M	27	9444112557	ravi_1@2
633470	ARUL	M	30	9444112555	arul@123

Figure 9. Employee Table and Sample Records

The customer can place an order with the correct customer ID and phone number as shown in Figure 10(a). If the customer ID is not correct, then it will be instructed to him by displaying a message at the bottom of the panel as shown in Figure 10(b). Similarly, if the phone number entered is not correct then that will also be instructed to him as shown in Figure 10(c). The orders placed by customers are shown in Figure 11.

Figure 10 consists of three screenshots of a web application window titled 'CUSTOMER WIND...' with a 'PLACE ORDER' button at the top. Each screenshot shows a form with the following fields: CUSTOMER ID, FOOD QUANTITY, ADDRESS, and PHONE NUMBER. Below these fields is a dropdown menu for 'Use Arrow Button & Select Food Item' and a 'Submit' button.

- (a) Correct ID & Phone: The CUSTOMER ID is 222170, FOOD QUANTITY is 3, ADDRESS is 'ndur Apt., Anna nagar', and PHONE NUMBER is 9444112579. The dropdown menu is set to 'IDLY'. The 'Submit' button is highlighted in blue. Below the button, the text 'SUCCESSFULLY ADDED' is displayed.
- (b) Invalid ID: The CUSTOMER ID is 195267, FOOD QUANTITY is 1, ADDRESS is '136, 6th Avenue', and PHONE NUMBER is 9444112551. The dropdown menu is set to 'DOSA'. Below the 'Submit' button, the message 'Invalid Customer ID. Please enter a valid ID.' is displayed.
- (c) Wrong Phone Number: The CUSTOMER ID is 195268, FOOD QUANTITY is 1, ADDRESS is '136, 6th Avenue', and PHONE NUMBER is 9444112552. The dropdown menu is set to 'DOSA'. Below the 'Submit' button, the message 'Invalid Customer ID. Please enter a valid ID. Enter registered cell phone number.' is displayed.

(a) Correct ID & Phone

(b) Invalid ID

(c) Wrong Phone Number

Figure 10. Placing Orders

```
mysql> SELECT * FROM Cust_order;
```

c_id	Food_name	Food_qty	Address	cphone
195268	DOSA	1	136, 6th Avenue	9444112551
222170	IDLY	3	1845, Chendur Apt., Anna nagar	9444112579

Figure 11. Cust_order (Customer Order) Table and Sample Records

After the customer places an order, it should be accepted by the restaurant manager using the panel shown in Figure 12. He has to choose the customer ID from the list displayed on the combo box, then he has to assign an employee to deliver the food by using the combo box provided at the bottom of the panel, and then he has to press the “Accept” button. A message “Order Accepted” will

be displayed in a message box as shown in Figure 12 to indicate order has been accepted successfully. The IDs of all the customers whose orders were accepted will be stored in a table as shown in Figure 13. The corresponding details of customers, and employees assigned for the orders accepted are stored in another table as shown in Figure 14.

The image shows a web application interface for managing orders. The main form, titled 'MANAGE ORDERS', has a dropdown menu for 'SELECT Customer ID of Order Received' with '195268' selected. Below this are input fields for 'Customer ID:', 'Customer Name:', 'Customer Phone:', 'Email:', 'Food ID:', 'Food Name:', 'Food Qty:', 'Price:', 'Employee ID:', 'Employee Phone:', and an 'Assign an Employee' dropdown with 'BABU' selected. A large red 'Accept Order' button is at the bottom. To the right, a smaller 'Restaurant Manager' window displays a blue information icon and the text 'Order Accepted' with an 'OK' button.

Figure 12. Selecting Order ID, Assigning Employee and Accepting Order by Restaurant Manager

```
mysql> SELECT * FROM Customer_orders;
+-----+-----+
| Order_index | Ordered_cust_ids |
+-----+-----+
| 1 | 195268 |
| 2 | 222170 |
+-----+-----+
```

Figure 13. Order IDs Stored in the Customer_orders Table

```
mysql> SELECT * FROM Rest_manageOrder;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| orderid | c_id | c_name | cphone | email | Food_id | Food_name | Food_qty | Price | Emp_id | ename | emp_phone |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 195268 | Nivedha | 9444112551 | nivedha@123 | 2 | DOSA | 1 | 100 | 158732 | BABU | 9444112556 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Figure 14. Accepted Orders Stored in the Rest_manageOrder Table

After the manager accepts the order, the customer may view his order by using the option “View Accepted Order” and by entering the correct customer ID as shown in Figure 15(a). If the order has not been accepted, then a message will be displayed in a message box as shown in Figure 15(b)

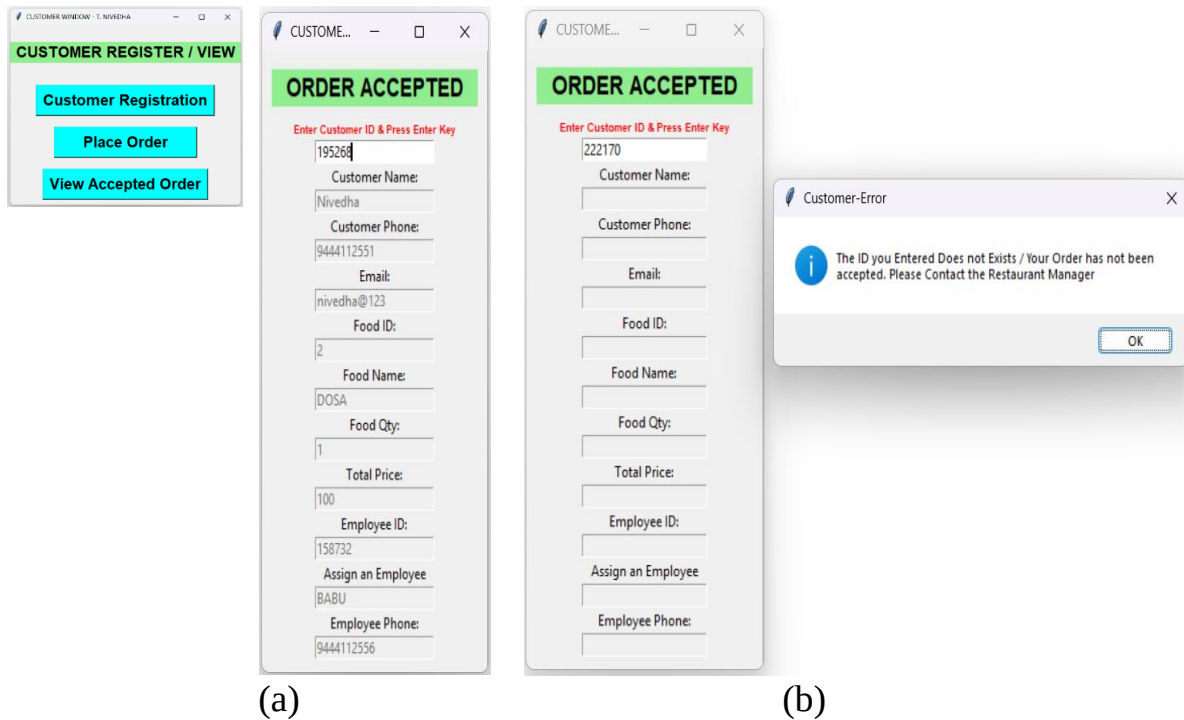
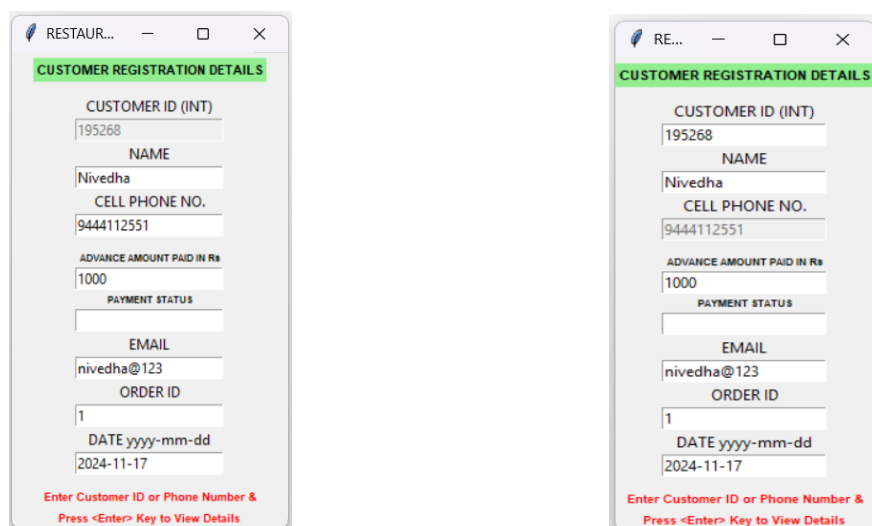


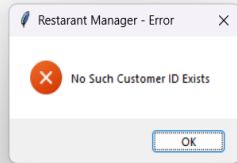
Figure 15. Accepted Orders Stored in the Rest_manageOrder Table



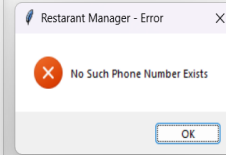
(a) If Customer ID is Known

(b) If Phone Number is Known

Figure 16. Customer Details Retrieved by the Restaurant Manager with Correct Customer ID or Phone Number

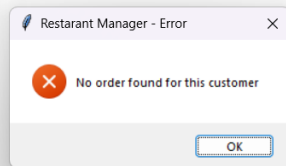


(a) By Giving Wrong Customer ID

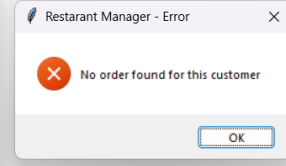


(b) By Giving Wrong Phone Number

Figure 17. Error Messages Displayed When the Restaurant Manager Tries to Retrieve Customer Details



(a) Customer ID Exists,
But No Order Placed



(b) Customer Phone Number Exists.
But No Order Placed

Figure 18. Error Messages Displayed When the Restaurant Manager Tries to Find out the Order Placed

The restaurant manager can retrieve the details of the customer either by giving the customer ID as shown in Figure 16(a) or by giving the phone number of the customer as shown in Figure 16 (b). If he gives a wrong ID, he will be shown an error message as given in Figure 17(a) or if he enters a wrong phone number he will be shown a message in a message box as given in Figure 17(b). If the ID and phone numbers of the customers are correct, but if the customer has not placed any order, then error messages will be displayed as shown in Figure 18.

4. SOURCE CODE

```
1  ""
2  Created on Mon Apr 29 16:25:08 2024
3
4  @author:T.NIVEDHA
5  ""
6  #
7
8  import mysql.connector
9  from tkinter import *
10 import tkinter.messagebox #provides message box
11 from tkinter import ttk #needed for combo box
12
13
14 mydb_con = mysql.connector.connect(host="localhost", user="root", password="Tnivedha@123")
15
16 #Creating DB
17
18 cur = mydb_con.cursor()
19 cur.execute("create database if not exists food1")
20 cur.execute("use food1")
21
22
23 ""
24 #If you make any changes in the table column names delete the database and enter the details again
25 #deleting database
26 sql = "DROP DATABASE food1"
27 #sql = "DROP TABLE Customer"
28 cur.execute(sql)
29 ""
30
31 Window = Tk()
32
33 # Define the Employee table
34 cur.execute("""
35 CREATE TABLE IF NOT EXISTS Employee (
36     Emp_id INT PRIMARY KEY,
37     ename VARCHAR(255) NOT NULL,
38     emp_g VARCHAR(10),
39     eage INT,
40     emp_phone BIGINT,
41     pwd VARCHAR(255)
42 )
43 """)
44
45 # Define the Customer table
46 cur.execute("""
47 CREATE TABLE IF NOT EXISTS Customer (
48     c_id INT PRIMARY KEY,
49     c_name VARCHAR(255) NOT NULL,
50     cphone BIGINT,
51     payment INT,
52     pstatus VARCHAR(20),
53     email VARCHAR(255),
54     orderid INT,
55     date DATE
56 )
57 """)
```

```

58
59
60 # Define the Food table  Food_id, Food_name, Food_quantity, Price_per_item
61 cur.execute("""
62 CREATE TABLE IF NOT EXISTS Food_list (
63     Food_id INT AUTO_INCREMENT PRIMARY KEY,
64     Food_name VARCHAR(255),
65     Food_quantity INT,
66     Price_per_item INT
67 )
68 """)
69
70 # Define the Rest_viewOrder table
71 cur.execute("""
72 CREATE TABLE IF NOT EXISTS Rest_manageOrder (
73     orderid INT AUTO_INCREMENT PRIMARY KEY,
74     c_id INT,
75     c_name VARCHAR(255) NOT NULL,
76     cphone BIGINT,
77     email VARCHAR(255),
78     Food_id INT,
79     Food_name VARCHAR(255),
80     Food_qty INT,
81     Price INT,
82     Emp_id INT,
83     ename VARCHAR(255),
84     emp_phone BIGINT
85 )
86 """)
87
88
89 # Define the Cust_order table
90 cur.execute("""
91 CREATE TABLE IF NOT EXISTS Cust_order (
92     c_id INT,
93     Food_name VARCHAR(255),
94     Food_qty INT,
95     Address VARCHAR(255),
96     cphone BIGINT
97 )
98 """)
99
100 # Store last order id.
101 cur.execute("""
102 CREATE TABLE IF NOT EXISTS Customer_orders (
103     Order_index INT AUTO_INCREMENT PRIMARY KEY,
104     Ordered_cust_ids INT  #only one customer id per index not ids
105 )
106 """)
107
108
109 #Generate a 6 digit customer ID that is not already in the table and return it as customer_id
110 def generate_unique_customer_id():
111     import random
112     #import string
113
114     while True:
115         # Generate a random 6-character ID
116         customer_id = random.randint(100000, 999999)
117

```



```

118     # Check if the ID exists in the database
119     sql = "SELECT * FROM Customer WHERE c_id = %s"
120     cur.execute(sql, (customer_id,))
121     result = cur.fetchone()
122
123     if not result:
124         return customer_id    #returns unique ID
125
126 #Generate a 6 digit Employee ID that is not already in the table and return it as customer_id
127 #Note that for each table different function names and table names are to be used
128 def generate_unique_employee_id():
129     import random
130     #import string
131
132     while True:
133         # Generate a random 6-character ID
134         employer_id = random.randint(100000, 999999)
135
136         # Check if the ID exists in the database
137         sql = "SELECT * FROM Employee WHERE Emp_id = %s"
138         cur.execute(sql, (employer_id,))
139         result = cur.fetchone()
140
141         if not result:
142             return employer_id #returns unique ID
143
144
145 def Employee_Register_OR_Show():
146
147     def Register_emp_details():
148         # Generate a unique employer ID
149         employer_id = generate_unique_employee_id()
150         #Insert the record for employer_id
151         Emp_data_insert = employer_id, e2.get(), e3.get(), e4.get(), e5.get(), e6.get() #get all entries
152         emp_sql = "INSERT INTO Employee (Emp_id, ename, emp_g, eage, emp_phone, pwd) VALUES (%s, %s, %s, %s, %s, %s)"
153         cur.execute(emp_sql, Emp_data_insert)
154         mydb_con.commit()
155         lbl_emp_added=Label(employee_frame,text="SUCCESSFULLY ADDED")
156         lbl_emp_added.pack()
157
158
159     def Show_emp_details():
160         e1_combo.config(state='normal')
161         cur.execute("SELECT Emp_id FROM Employee")
162         All_Employee_Ids = [row[0] for row in cur.fetchall()]
163         #fetches all records and takes only the first value in each record and puts in a list
164         e1_combo.config(values=All_Employee_Ids)    # Populate the combobox
165
166     def on_Emp_id_selected(event):
167         selected_Emp_id = e1_combo.get()
168         sql = "SELECT * FROM Employee WHERE Emp_id = %s"
169         cur.execute(sql, (selected_Emp_id,))
170         Employee_record = cur.fetchone()
171         e2.delete(0,END) #blank the input widgets
172         e3.delete(0,END)
173         e4.delete(0,END)
174         e5.delete(0,END)
175         e6.delete(0,END)
176         # Populate the other fields with the retrieved result data
177         e2.insert(0, Employee_record[1]) # Food name

```

```

178         e3.insert(0, Employee_record[2]) # Food quantity
179         e4.insert(0, Employee_record[3]) # Price per item
180         e5.insert(0, Employee_record[4]) # Food name
181         e6.insert(0, Employee_record[5]) # Food quantity
182
183
184         e1_combo.bind("<<ComboboxSelected>>", on_Emp_id_selected)
185         # should be put only after defining the event function. But may not be moved out of Show_emp_details()
186
187     #function code starts
188     employee_frame = Toplevel(Window)
189     label1=Label(employee_frame,text="EMPLOYEE DETAILS",font='arial 15 bold', bg='light blue')
190     label1.pack(pady=(5,10))
191     label_instruction1=Label(employee_frame,text="Press Ctrl and v keys simultaneously",font='arial 8 bold', fg='red')
192     label_instruction1.pack()
193     label_instruction2=Label(employee_frame,text="to change to view mode",font='arial 8 bold', fg='red')
194     label_instruction2.pack()
195
196     l1=Label(employee_frame,text="EMPLOYEE ID")
197     l1.pack()
198
199     # Create a hidden combobox
200     e1_combo = ttk.Combobox(employee_frame)
201     e1_combo.pack(pady=(0,15))
202     e1_combo.config(state='disabled')
203
204
205     l2=Label(employee_frame,text="EMPLOYEE NAME")
206     l2.pack()
207     e2=tkinter.Entry(employee_frame)
208     e2.pack()
209     l3=Label(employee_frame,text="EMPLOYEE GENDER")
210     l3.pack()
211     e3=tkinter.Entry(employee_frame)
212     e3.pack()
213     l4=Label(employee_frame,text="EMPLOYEE AGE")
214     l4.pack()
215     e4=tkinter.Entry(employee_frame)
216     e4.pack()
217     l5=Label(employee_frame,text="EMPLOYEE PHONE NO:")
218     l5.pack()
219     e5=tkinter.Entry(employee_frame)
220     e5.pack()
221     l6=Label(employee_frame,text="ENTER PASS WORD")
222     l6.pack()
223     e6=tkinter.Entry(employee_frame)
224     e6.pack()
225
226
227     but_emp_register=Button(employee_frame,text="Register",font="arial 20 bold",bg='cyan',command=lambda: Register_emp_details())
228     but_emp_register.pack(pady=20)
229     #but_emp_show=Button(employee_frame,text="View",font="arial 20 bold",bg='cyan',command=lambda: Show_emp_details())
230     #but_emp_show.pack(side=RIGHT, pady=10)
231     def activate_view_mode(event): #Ctrl+v i.e on_ctrl_v event is attached to the Add_food_frame
232         but_emp_register.config(state='disabled')
233         #lbl_successfully_added.destroy()
234         Show_emp_details()
235
236     employee_frame.bind("<Control-v>", activate_view_mode)
237
238

```

```

239
240 def Customer_Register_ShowDetails(Cutomer_OR_Manager):
241
242     def Register_cust_details():
243         # Generate a unique customer ID
244         customer_id = generate_unique_customer_id()
245         but_cust_store['state'] = DISABLED
246         #c1 & c7 widgets are not used while registration
247         Cust_data_insert = customer_id, c2.get(), c3.get(), c4.get(), c5.get(), c6.get(), c8.get()
248         cust_sql = "INSERT INTO Customer (c_id, c_name, cphone, payment, pstatus, email, date) VALUES (%s, %s, %s, %s, %s, %s, %s)"
249         cur.execute(cust_sql, Cust_data_insert)
250         mydb_con.commit()
251         lbl_cust_added=Label(customer_frame,text="SUCCESSFULLY ADDED")
252         lbl_cust_added.pack()
253         c1['state'] = NORMAL
254         c1.insert(0,customer_id)
255         c1['state'] = DISABLED
256         custID_str= "Your Cutomer ID is  " + str(customer_id) + "  Please Store it Safly"
257         #lbl_cust_added=Label(customer_frame,text=custID_str)
258         #lbl_cust_added.pack()
259         messagebox.showinfo("Customer Registration", custID_str, parent=customer_frame)
260         Window.destroy()
261
262     def Show_cust_details(Cust_rec): #Pass the customer record retrived based on customer ID or phone number
263         c2['state'] = NORMAL
264         c4['state'] = NORMAL
265         c5['state'] = NORMAL
266         c6['state'] = NORMAL
267         c8['state'] = NORMAL
268         #c1.insert(0, Cust_rec[0][0]) # even c_id is the primary key it is to be inserted in the entry widget
269         c2.insert(0, Cust_rec[1])
270         #inserts the text in the input widget, first index is the record number and the second index is the item
271         c4.insert(0, Cust_rec[3])
272         c5.insert(0, Cust_rec[4])
273         c6.insert(0, Cust_rec[5])
274         c8.insert(0, Cust_rec[7]) # date is inserted and then order_id is checked for availability
275
276
277
278 # function code starts
279 customer_frame = Toplevel(Window)
280 labelc=Label(customer_frame,text="CUSTOMER REGISTRATION DETAILS",font='arial 8 bold', bg='light green')
281 labelc.pack(pady=(5,10))
282
283 l1=Label(customer_frame,text="CUTOMER ID (INT)")
284 l1.pack()
285 c1=tkinter.Entry(customer_frame)
286 c1.pack()
287 c1.config(state='disabled')
288
289
290 l2=Label(customer_frame,text="NAME")
291 l2.pack()
292 c2=tkinter.Entry(customer_frame)
293 c2.pack()
294 l3=Label(customer_frame,text="CELL PHONE NO.")
295 l3.pack()
296 c3=tkinter.Entry(customer_frame)
297 c3.pack()
298 l4=Label(customer_frame,text="ADVANCE AMOUNT PAID IN Rs", font='arial 6 bold')

```

```

299     l4.pack(pady=(10,0))
300     c4=tkinter.Entry(customer_frame)
301     c4.pack()
302     l5=Label(customer_frame,text="PAYMENT STATUS", font='arial 6 bold')
303     l5.pack()
304     c5=tkinter.Entry(customer_frame)
305     c5.pack()
306     c5['state'] = DISABLED
307     l6=Label(customer_frame,text="EMAIL")
308     l6.pack()
309     c6=tkinter.Entry(customer_frame)
310     c6.pack()
311     l7=Label(customer_frame,text="ORDER ID")
312     # disable orderid during registartion and show while viewing, 7th item created in Rest_view_order table
313     l7.pack()
314     c7=tkinter.Entry(customer_frame)
315     c7.pack()
316     l8=Label(customer_frame,text="DATE yyyy-mm-dd")
317     l8.pack()
318     c8=tkinter.Entry(customer_frame)
319     c8.pack()
320
321
322     def check_entry_c1_and_process():
323         #This event function gets the customer ID from the Entry Widget and displays other details
324         Cust_id=c1.get()
325         c1["state"]=DISABLED
326         sql = "SELECT * FROM Customer WHERE c_id = %s"
327         cur.execute(sql, (Cust_id,))
328         Cust_rec = cur.fetchone()
329
330         if Cust_rec:
331             # Proceed with fetching the order ID only if customer record exists
332             accepted_order_id_sql = "SELECT orderid FROM Rest_manageOrder WHERE c_id = %s"
333             cur.execute(accepted_order_id_sql, (Cust_id,))
334             order_id_fetched = cur.fetchone()
335             # Check if an order ID was found
336             if order_id_fetched:
337                 # Process the order_id_fetched if available
338                 c3['state'] = NORMAL
339                 c3.insert(0, Cust_rec[2]) #Display the phone number
340                 c7['state'] = NORMAL
341                 c7.insert(0, order_id_fetched[0])
342                 #if orderid 7th item is created by the Manager in the Cust_rect_view_order table then show it
343                 Show_cust_details(Cust_rec) #Display other other
344             else:
345                 messagebox.showerror("Restarant Manager - Error", "No order found for this customer")
346                 Window.destroy()
347         else:
348             messagebox.showerror("Restarant Manager - Error", "No Such Customer ID Exists")
349             Window.destroy()
350
351
352
353
354     def check_entry_c3_and_process(event):
355         #This event function gets the customer Phone number from the Entry Widget and displays other details
356         Phone_no=c3.get()
357         c3["state"]=DISABLED
358         sql = "SELECT * FROM Customer WHERE cphone = %s"

```

```

359 cur.execute(sql, (Phone_no,))
360 Cust_rec = cur.fetchone() #None data type will be returned when the table is empty. Causes error
361
362 if Cust_rec:
363     # Proceed with fetching the order ID only if customer record exists
364     accepted_order_phone_sql = "SELECT orderid FROM Rest_manageOrder WHERE cphone = %s"
365     cur.execute(accepted_order_phone_sql, (Phone_no,))
366     order_phone_id_fetched = cur.fetchone()
367     # Check if an order ID was found
368     if order_phone_id_fetched:
369         # Process the order_id_fetched if available
370         c1['state'] = NORMAL
371         c1.insert(0, Cust_rec[0]) #Display the customer id
372         c7['state'] = NORMAL
373         c7.insert(0, order_phone_id_fetched[0])
374         #if orderid 7th item is order_phone_id_fetched by the Manager in the Cust_rect_view_order table then show it
375         Show_cust_details(Cust_rec) #Display other other
376     else:
377         messagebox.showerror("Restarant Manager - Error", "No order found for this customer")
378         Window.destroy()
379 else:
380     messagebox.showerror("Restarant Manager - Error", "No Such Customer ID Exists")
381     Window.destroy()
382
383
384 # Bind the Enter key to the check_entry_and_process function
385 #Two ways to use event function. The label event should be passed to check_entry_c3_and_process() function in the second case
386 c1.bind('<Return>', lambda event: check_entry_c1_and_process()) #retrives details using customer ID
387 c3.bind('<Return>', check_entry_c3_and_process) #retrieves details using customer phone number
388
389
390 if Cutomer_OR_Manager == 1: #Register
391     but_cust_store=Button(customer_frame,text="Register",font="arial 20 bold",bg='cyan',command=lambda: Register_cust_details())
392     but_cust_store.pack(pady=20)
393     # disable orderid during registartion and bring while viewing from orderid 7th item created in Rest_view_order table
394     c7['state'] = DISABLED
395 elif Cutomer_OR_Manager == 2: # View Customer Details
396     c1['state'] = NORMAL
397     c1.delete(0,END)
398     c2.delete(0,END)
399     c3['state'] = NORMAL
400     c3.delete(0,END) #phone number
401     c4.delete(0,END)
402     c5.delete(0,END)
403     c6.delete(0,END)
404     #c7['state'] = NORMAL
405     c7.delete(0,END)
406     c8.delete(0,END)
407
408     c2['state'] = DISABLED
409     #c3['state'] = DISABLED
410     c4['state'] = DISABLED
411     c5['state'] = DISABLED
412     c6['state'] = DISABLED
413     c7['state'] = DISABLED
414     c8['state'] = DISABLED
415
416
417 cust_l1=Label(customer_frame,text="Enter Customer ID or Phone Number &", font="arial 7 bold", fg='red')
418 #Pad 10 pixels above and 5 pixels below
419 cust_l1.pack(pady=(10,1))

```

```

420     cust_l2=Label(customer_frame,text="Press <Enter> Key to View Details", font="arial 7 bold", fg='red')
421     cust_l2.pack()
422
423
424 def Add_food_list():
425
426     def Store_food_details():
427         # disable orderid during registration and bring while viewing from orderid 7th item created in Rest_view_order table
428         #Instead of fe1 a combo button is used for food-id and it is the Primary key & auto incremented, need not be included
429         Food_data =fe2.get(), fe3.get(), fe4.get()
430         #food_sql = "INSERT INTO Food_list (Food_id, Food_name, Food_quantity, Price_per_item) VALUES (%s, %s, %s, %s)"
431         food_sql = "INSERT INTO Food_list (Food_name, Food_quantity, Price_per_item) VALUES (%s, %s, %s)"
432         cur.execute(food_sql, Food_data)
433         mydb_con.commit()
434         lbl_successfully_added=Label(Add_food_frame,text="SUCCESSFULLY ADDED")
435         lbl_successfully_added.pack()
436
437     def Show_food_details():
438         food_id_combo.config(state='normal')
439         cur.execute("SELECT Food_id FROM Food_list")
440         food_ids = [row[0] for row in cur.fetchall()] #feteches all records and takes only the first value in each record and puts in a list
441         food_id_combo.config(values=food_ids)      # Populate the combobox
442
443     def on_food_id_selected(event):
444         selected_food_id = food_id_combo.get()
445         sql = "SELECT Food_name, Food_quantity, Price_per_item FROM Food_list WHERE Food_id = %s"
446         cur.execute(sql, (selected_food_id,))
447         Food_record = cur.fetchone()
448         fe2.delete(0,END) #blank the input widgets otherwise contents will be concatenated
449         fe3.delete(0,END)
450         fe4.delete(0,END)
451         # Populate the other fields with the retrieved result data
452         fe2.insert(0, Food_record[0]) # Food name
453         fe3.insert(0, Food_record[1]) # Food quantity
454         fe4.insert(0, Food_record[2]) # Price per item
455
456         food_id_combo.bind("<<ComboboxSelected>>", on_food_id_selected)
457
458
459     Add_food_frame = Toplevel(Window)
460     # Add_food_frame is an instance of window associated with the main window
461
462     def activate_view_mode(event):    #Ctrl+v i.e on_ctrl_v event is attached to the Add_food_frame
463         b1.config(state='disabled')
464         fe2.delete(0,END) #blank the input widgets. Otherwise contents will be concatenated
465         fe3.delete(0,END)
466         fe4.delete(0,END)
467         Show_food_details()
468
469     Add_food_frame.bind("<Control-v>", activate_view_mode)
470
471
472     frame_lbl = Label(Add_food_frame, text="ADD FOOD TO THE LIST", font="arial 15 bold", bg='light blue')
473     frame_lbl.pack()
474
475     instruction_label = Label(Add_food_frame, text="Click on the Window & Press Ctrl and v", font='arial 8 bold', fg='red')
476     instruction_label.pack(pady=(10,0))
477     instruction_labe2 = Label(Add_food_frame, text="together to change to View mode", font='arial 8 bold', fg='red')
478     instruction_labe2.pack(pady=(0,0))
479
480     flbl1 = Label(Add_food_frame, text="FOOD ID")

```

```

481 flbl1.pack()
482
483 # Create a hidden combobox
484 food_id_combo = ttk.Combobox(Add_food_frame)
485 food_id_combo.pack(pady=(0,10))
486 food_id_combo.config(state='disabled')
487
488 flbl2 = Label(Add_food_frame, text="FOOD NAME")
489 flbl2.pack()
490 fe2 = tkinter.Entry(Add_food_frame)
491 fe2.pack()
492 flbl3 = Label(Add_food_frame, text="FOOD QUANTITY")
493 flbl3.pack()
494 fe3 = tkinter.Entry(Add_food_frame)
495 fe3.pack()
496 flbl4 = Label(Add_food_frame, text="PRICE PER ITEM")
497 flbl4.pack()
498 fe4 = tkinter.Entry(Add_food_frame)
499 fe4.pack()
500
501 b1=Button(Add_food_frame, text="Store",font="arial 20 bold",bg='cyan',command=lambda: Store_food_details())
502 b1.pack(pady=30)
503
504
505
506 def Cust_order():
507     def populate_combo():
508         combolbl = Label(Cust_order_frame, text=" Use Arrow Button & Select Food Item ", font="arial 10 bold", fg='red')
509         combolbl.pack(pady=(10,0)) #pad from previous widget
510         food_combo = ttk.Combobox(Cust_order_frame)
511         food_combo.pack(pady=(0,10))
512         # Bind the add_food_item function to the Enter key press in the entry widget (using fentry)
513         #fe1.bind("<Return>", lambda event: add_food_item(fe1, food_combo))
514         # Retrieve food names
515         sql = "SELECT Food_name FROM Food_list"
516         cur.execute(sql)
517         food_names = [row[0] for row in cur.fetchall()] # Get food names from results
518
519         # Clear existing items (optional)
520         food_combo.set("")
521         food_combo['values'] = food_names
522         # Bind selection event to update oe2 entry
523         food_combo.bind("<<ComboboxSelected>>", lambda event: update_oe2(food_combo.get()))
524
525     def update_oe2(selected_food):
526         oe2.delete(0, END) # Clear existing text in oe2 entry
527         oe2.insert(0, selected_food) # Insert the selected food item
528
529     def Submit_food_order():
530         # Get customer ID and phone number from entry widgets
531         customer_id = oe1.get()
532         customer_phone = oe5.get()
533         # Validate customer ID by checking against Customer table
534         sql = "SELECT cphone FROM Customer WHERE c_id = %s"
535         cur.execute(sql, (customer_id,))
536         registered_phone = cur.fetchone() # Fetch first row (if exists). This will return 0 if the record does not exist for the c_id
537
538         if not registered_phone: #This is not checking the phone no. It checks whether the record exists. if record does not exist for the c_id
539             # Display error message for invalid customer ID
540             lbl_error = Label(Cust_order_frame, text="Invalid Customer ID. Please enter a valid ID.", fg='red')
541             lbl_error.pack()

```

```

542         return # Exit the function without submitting the order
543
544     # Validate phone number by comparing with registered phone
545     if int(customer_phone) != registered_phone[0]: #customer_phone is astring and registered_phone[0] is an integer
546         # Display error message for mismatched phone number
547         lbl_error = Label(Cust_order_frame, text="Enter registered cell phone number.", fg='red')
548         lbl_error.pack()
549         return # Exit the function without submitting the order
550
551
552     # disable orderid during registration and bring while viewing from orderid 7th item created in Rest_view_order table
553     Food_data = oe1.get(), oe2.get(), oe3.get(), oe4.get(), oe5.get() #fe1.get(), is Primary key & auto incremented, need not be included
554     food_sql = "INSERT INTO Cust_order (c_id, Food_name, Food_qty, Address, cphone) VALUES (%s, %s, %s, %s, %s)"
555     cur.execute(food_sql, Food_data)
556     mydb_con.commit()
557     lbl_successfully_added=Label(Cust_order_frame,text="SUCCESSFULLY ADDED")
558     lbl_successfully_added.pack()
559     custID_from_order = oe1.get() # custID_from_order - set by Cust_order() function is saved
560     order_sql = "INSERT INTO Customer_orders (Ordered_cust_ids) VALUES (%s)"
561     cur.execute(order_sql, (custID_from_order,))
562     mydb_con.commit()
563
564     def Show_food_order():
565
566         Co_id = int(oe1.get())
567         sql = "SELECT * FROM Cust_order WHERE c_id = %s"
568         cur.execute(sql, (Co_id ,)) #c_id_from_order received from the Customer Order window widget
569         res = cur.fetchall() #None' data type will be returned when the table is empty. Causes error
570         oe1.delete(0,END)
571         oe2.delete(0,END) #inserts the text in the input widget
572         oe3.delete(0,END)
573         oe4.delete(0,END)
574         oe5.delete(0,END)
575         oe1.insert(0, res[0][0]) # even c_id is the primary key it is to be inserted in the entry widget
576         oe2.insert(0, res[0][1]) #inserts the text in the input widget, first index is the record number and the second index is the item
577         oe3.insert(0, res[0][2])
578         oe4.insert(0, res[0][3])
579         oe5.insert(0, res[0][4])
580
581     Cust_order_frame = Toplevel(Window)
582     frame_lbl = Label(Cust_order_frame, text=" PLACE ORDER ", font="arial 20 bold", bg='light green')
583     frame_lbl.pack(pady=(5,10))
584
585     olb1 = Label(Cust_order_frame, text="CUSTOMER ID")
586     olb1.pack()
587     oe1 = tkinter.Entry(Cust_order_frame)
588     oe1.pack()
589     olb2 = Label(Cust_order_frame, text="Use Combobox to Select the Food Item", font="arial 8 bold", fg='red')
590     olb2.pack(pady=(5,0))
591     oe2 = tkinter.Entry(Cust_order_frame)
592     oe2.pack()
593     olb3 = Label(Cust_order_frame, text="FOOD QUANTITY")
594     olb3.pack(pady=(10,0))
595     oe3 = tkinter.Entry(Cust_order_frame)
596     oe3.pack()
597     olb4 = Label(Cust_order_frame, text="ADDRESS")
598     olb4.pack()
599     oe4 = tkinter.Entry(Cust_order_frame)
600     oe4.pack()
601     olb5 = Label(Cust_order_frame, text="PHONE NUMBER")
602     olb5.pack()

```



```

603     oe5 = tkinter.Entry(Cust_order_frame)
604     oe5.pack()
605
606     populate_combo()
607
608     b1=Button(Cust_order_frame, text="Submit",font="arial 20 bold",bg='cyan',command=lambda: Submit_food_order())
609     b1.pack(pady=(15,5)) #Provides 15 pixels padding above the button to display combo box content
610
611
612 def Rest_manageOrder():
613
614     def populate_combo_employee():
615         # Get all employee names
616         sql = "SELECT ename FROM Employee"
617         cur.execute(sql)
618         employee_names = [row[0] for row in cur.fetchall()] # Check if index is ename (row[0])
619         # Clear existing items (optional)
620         assign_emp_combo.set("")
621         assign_emp_combo['values'] = employee_names
622
623     def select_employee(event): #called when an item is selected in the combo box
624         selected_emp = assign_emp_combo.get()
625         # Get employee details based on selected name
626         sql = "SELECT Emp_id, emp_phone FROM Employee WHERE ename = %s"
627         cur.execute(sql, (selected_emp,))
628         emp_data = cur.fetchone()
629
630         if emp_data:
631             emp_id_entry.delete(0, END)
632             emp_id_entry.insert(0, emp_data[0])
633             emp_phone_entry.delete(0, END)
634             emp_phone_entry.insert(0, emp_data[1])
635
636
637     def poulate_orderCombo():
638         order_sql = "SELECT Ordered_cust_ids FROM Customer_orders"
639         cur.execute(order_sql)
640         all_orders_cust_ids= [row[0] for row in cur.fetchall()]
641         # Clear existing items (optional)
642         orders_combo.set("")
643         orders_combo['values'] = all_orders_cust_ids
644
645
646     def assign_selected_value(orders_cust_id_selected): #event function called when an item is selected on the orders_combo
647         global selected_order_cust_id # Use global keyword to modify the variable outside the function
648         selected_order_cust_id = orders_cust_id_selected
649         select_order_andAccept() # get the values for other widgets from the tables and enter into the widgets
650
651
652     def select_order_andAccept():
653
654         sql = "SELECT c_name,cphone, email FROM Customer WHERE c_id = %s"
655         cur.execute(sql, (selected_order_cust_id,))
656         # selected_order_cust_id defined as global in assign_selected_value() event function of orders_combo
657         customer_data = cur.fetchone()
658         if customer_data:
659             c_id_entry.delete(0, END)
660             c_id_entry.insert(0,selected_order_cust_id) #customer id brought from the order combo box
661             c_name_entry.delete(0, END)
662             c_name_entry.insert(0, customer_data[0])

```

```

663         c_phone_entry.delete(0, END)
664         c_phone_entry.insert(0, customer_data[1])
665         email_entry.delete(0, END)
666         email_entry.insert(0, customer_data[2])
667
668
669         sql = "SELECT Food_name, Food_qty FROM Cust_order WHERE c_id = %s"
670         cur.execute(sql, (selected_order_cust_id,))
671         # selected_order_cust_id defined as global in assign_selected_value() event function of orders_combo
672         order_details = cur.fetchone()
673
674         if order_details:
675             # Set retrieved order details
676             food_name_entry.delete(0, END)
677             food_name_entry.insert(0, order_details[0])
678             food_qty_entry.delete(0, END)
679             food_qty_entry.insert(0, order_details[1])
680
681         #From the above Food_name order_details[0], get Food_id, price from Food_list table (Primary key)
682
683         sql = "SELECT Food_id, price_per_item FROM Food_list WHERE Food_name = %s"
684         cur.execute(sql, (order_details[0],)) # order_details[0] = Food_id
685         Food_id_price = cur.fetchone()
686
687         if Food_id_price:
688             Total_price=Food_id_price[1]*order_details[1] # order_details[2] = Food_qty
689             price_entry.delete(0,END)
690             price_entry.insert(0>Total_price)
691             food_id_entry.delete(0, END)
692             food_id_entry.insert(0, Food_id_price[0])
693
694     def Accept_order():
695
696         manager_data = (
697             c_id_entry.get(),
698             c_name_entry.get(),
699             c_phone_entry.get(),
700             email_entry.get(),
701             food_id_entry.get(),
702             food_name_entry.get(),
703             food_qty_entry.get(),
704             price_entry.get(),
705             emp_id_entry.get(),
706             assign_emp_combo.get(), #employee name assigned by the manager
707             emp_phone_entry.get(),
708         )
709         manager_sql = "INSERT INTO Rest_manageOrder (c_id,c_name,cphone,email,Food_id,Food_name,Food_qty,Price,
710 Emp_id,ename,emp_phone) VALUES %s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)"
711         cur.execute(manager_sql, manager_data)
712         mydb_con.commit()
713         #orderAccept_lbl = Label(Manage_order_frame, text="ORDER ACCEPTED", font="arial 10 bold", bg='light blue')
714         #orderAccept.pack()
715         messagebox.showinfo("Restaurant Manager", "Order Accepted", parent=Manage_order_frame)
716
717     # Create the window and frame
718     Manage_order_window = Toplevel()
719     Manage_order_window.title("Manage Order")
720     Manage_order_frame = Frame(Manage_order_window)
721     Manage_order_frame.pack(padx=10, pady=10)
722     frame_lbl = Label(Manage_order_frame, text=" MANAGE ORDERS ", font="arial 15 bold", bg='light blue')
723     frame_lbl.pack(pady=(5,10))

```

```

724     combolbl = Label(Manage_order_frame, text="SELECT Cutomer ID of Order Received", font='arial 7 bold', fg='red')
725     combolbl.pack(pady=(5,0)) #pad from previous widget
726     orders_combo = ttk.Combobox(Manage_order_frame)
727     orders_combo.bind("<<ComboboxSelected>>", lambda event: assign_selected_value(orders_combo.get()))
728     orders_combo.pack()
729
730
731
732 # Customer Details
733     c_id_lbl = Label(Manage_order_frame, text="Customer ID:")
734     c_id_lbl.pack()
735     c_id_entry = Entry(Manage_order_frame)
736     c_id_entry.pack()
737
738     c_name_lbl = Label(Manage_order_frame, text="Customer Name:")
739     c_name_lbl.pack()
740     c_name_entry = Entry(Manage_order_frame)
741     c_name_entry.pack()
742
743     c_phone_lbl = Label(Manage_order_frame, text="Customer Phone:")
744     c_phone_lbl.pack()
745     c_phone_entry = Entry(Manage_order_frame)
746     c_phone_entry.pack()
747
748     email_lbl = Label(Manage_order_frame, text="Email:")
749     email_lbl.pack()
750     email_entry = Entry(Manage_order_frame)
751     email_entry.pack()
752
753 # Order Details
754     food_id_lbl = Label(Manage_order_frame, text="Food ID:")
755     food_id_lbl.pack()
756     food_id_entry = Entry(Manage_order_frame)
757     food_id_entry.pack()
758
759     food_name_lbl = Label(Manage_order_frame, text="Food Name:")
760     food_name_lbl.pack()
761     food_name_entry = Entry(Manage_order_frame)
762     food_name_entry.pack()
763
764     food_qty_lbl = Label(Manage_order_frame, text="Food Qty:")
765     food_qty_lbl.pack()
766     food_qty_entry = Entry(Manage_order_frame)
767     food_qty_entry.pack()
768
769     price_lbl = Label(Manage_order_frame, text="Price:")
770     price_lbl.pack()
771     price_entry = Entry(Manage_order_frame)
772     price_entry.pack()
773
774 # Employee Details
775     emp_id_lbl = Label(Manage_order_frame, text="Employee ID:")
776     emp_id_lbl.pack()
777     emp_id_entry = Entry(Manage_order_frame)
778     emp_id_entry.pack()
779
780     emp_phone_lbl = Label(Manage_order_frame, text="Employee Phone:")
781     emp_phone_lbl.pack()
782     emp_phone_entry = Entry(Manage_order_frame)
783     emp_phone_entry.pack()

```

```

784
785     ename_lbl = Label(Manage_order_frame, text="Assign an Employee")
786     ename_lbl.pack()
787     emp_name_entry = Entry(Manage_order_frame)
788     emp_phone_entry.pack()
789
790     # Create combo button for employee selection
791     assign_emp_combo = ttk.Combobox(Manage_order_frame)
792     assign_emp_combo.bind("<<ComboboxSelected>>", select_employee)
793     assign_emp_combo.pack()
794
795     b3=Button(Manage_order_frame, text="Accept Order",font="arial 20 bold",bg='cyan',command=lambda: Accept_order())
796     b3.pack(pady=10)
797
798     poulate_orderCombo()
799     populate_combo_employee() #called in the Rest_manageOrder() function
800
801
802
803 def CutomerView_AcceptedOrder():
804     # Create the window and frame
805     View_Accepted_order_window = Toplevel()
806     View_Accepted_order_window.title("CUSTOMER ORDER ACCEPTED")
807     View_Accepted_order_frame = Frame(View_Accepted_order_window)
808     View_Accepted_order_frame.pack(padx=10, pady=10)
809     frame_lbl = Label(View_Accepted_order_frame, text=" ORDER ACCEPTED ", font="arial 15 bold", bg='light green')
810     frame_lbl.pack(pady=(5,10))
811
812
813 def get_c_id_entry(event):
814     """
815     This function is called when the Enter key is pressed in the c_id_entry field.
816     It retrieves the entered value and (you can replace this with your desired action)
817     prints it to the console.
818     """
819     customer_id = c_id_entry.get()
820     sql = "SELECT c_name,cphone,email,Food_id,Food_name,Food_qty,Price,Emp_id,ename,emp_phone FROM Rest_manageOrder WHERE c_id = %s"
821     cur.execute(sql, (customer_id,))
822     # selected_order_cust_id defined as global in assign_selected_value() event function of orders_combo
823     accepted_order_details = cur.fetchone()
824
825     if accepted_order_details:
826         #Enable the widgets and clear the contents
827         c_name_entry['state']= NORMAL
828         c_phone_entry['state']= NORMAL
829         email_entry['state']= NORMAL
830         food_id_entry['state']= NORMAL
831         food_name_entry['state']= NORMAL
832         food_qty_entry['state']= NORMAL
833         price_entry['state']= NORMAL
834         emp_id_entry['state']= NORMAL
835         emp_name_entry['state']= NORMAL
836         emp_phone_entry['state']= NORMAL
837
838         #clear all widgets
839         c_name_entry.delete(0, END)
840         c_phone_entry.delete(0, END)
841         email_entry.delete(0, END)
842         food_id_entry.delete(0, END)
843         food_name_entry.delete(0, END)
844         food_qty_entry.delete(0, END)

```

```

845     price_entry.delete(0,END)
846     emp_id_entry.delete(0, END)
847     emp_name_entry.delete(0, END)
848     emp_phone_entry.delete(0, END)
849     #Insert / display details retrived from the database into the widgeta
850     c_name_entry.insert(0, accepted_order_details[0])
851     c_phone_entry.insert(0, accepted_order_details[1])
852     email_entry.insert(0, accepted_order_details[2])
853     food_id_entry.insert(0, accepted_order_details[3])
854     food_name_entry.insert(0, accepted_order_details[4])
855     food_qty_entry.insert(0, accepted_order_details[5])
856     price_entry.insert(0,accepted_order_details[6])
857     emp_id_entry.insert(0, accepted_order_details[7])
858     emp_name_entry.insert(0, accepted_order_details[8])
859     emp_phone_entry.insert(0, accepted_order_details[9])
860     #Disable the widgets to avoid entering details into those widgets
861     c_name_entry['state']= DISABLED
862     c_phone_entry['state']= DISABLED
863     email_entry['state']= DISABLED
864     food_id_entry['state']= DISABLED
865     food_name_entry['state']= DISABLED
866     food_qty_entry['state']= DISABLED
867     price_entry['state']= DISABLED
868     emp_id_entry['state']= DISABLED
869     emp_name_entry['state']= DISABLED
870     emp_phone_entry['state']= DISABLED
871
872 else:
873     #Enable the widgets and clear the contes
874     c_name_entry['state']= NORMAL
875     c_phone_entry['state']= NORMAL
876     email_entry['state']= NORMAL
877     food_id_entry['state']= NORMAL
878     food_name_entry['state']= NORMAL
879     food_qty_entry['state']= NORMAL
880     price_entry['state']= NORMAL
881     emp_id_entry['state']= NORMAL
882     emp_name_entry['state']= NORMAL
883     emp_phone_entry['state']= NORMAL
884
885     #clear the contents
886     c_name_entry.delete(0, END)
887     c_phone_entry.delete(0, END)
888     email_entry.delete(0, END)
889     food_id_entry.delete(0, END)
890     food_name_entry.delete(0, END)
891     food_qty_entry.delete(0, END)
892     price_entry.delete(0,END)
893     emp_id_entry.delete(0, END)
894     emp_name_entry.delete(0, END)
895     emp_phone_entry.delete(0, END)
896
897     #Disbale the widgets
898     c_name_entry['state']= DISABLED
899     c_phone_entry['state']= DISABLED
900     email_entry['state']= DISABLED
901     food_id_entry['state']= DISABLED
902     food_name_entry['state']= DISABLED
903     food_qty_entry['state']= DISABLED
904     price_entry['state']= DISABLED

```

```

905     emp_id_entry['state']= DISABLED
906     emp_name_entry['state']= DISABLED
907     emp_phone_entry['state']= DISABLED
908
909     messagebox.showinfo("Customer-Error", "The ID you Entered Does not Exists. Please Enter the Correct Customer ID or Contact the Restaurant Manager")
910     Window.destroy()
911
912
913     c_id_lbl = Label(View_Accepted_order_frame, text="Enter Customer ID & Press Enter Key", font="arial 7 bold",fg='red')
914     c_id_lbl.pack()
915     c_id_entry = Entry(View_Accepted_order_frame)
916     # Bind the <Return> key event to the get_customer_id function
917     c_id_entry.bind("<Return>", get_c_id_entry)
918     c_id_entry.pack()
919
920     c_name_lbl = Label(View_Accepted_order_frame, text="Customer Name:")
921     c_name_lbl.pack()
922     c_name_entry = Entry(View_Accepted_order_frame)
923     c_name_entry.pack()
924
925     c_phone_lbl = Label(View_Accepted_order_frame, text="Customer Phone:")
926     c_phone_lbl.pack()
927     c_phone_entry = Entry(View_Accepted_order_frame)
928     c_phone_entry.pack()
929
930     email_lbl = Label(View_Accepted_order_frame, text="Email:")
931     email_lbl.pack()
932     email_entry = Entry(View_Accepted_order_frame)
933     email_entry.pack()
934
935     # Order Details
936     food_id_lbl = Label(View_Accepted_order_frame, text="Food ID:")
937     food_id_lbl.pack()
938     food_id_entry = Entry(View_Accepted_order_frame)
939     food_id_entry.pack()
940
941     food_name_lbl = Label(View_Accepted_order_frame, text="Food Name:")
942     food_name_lbl.pack()
943     food_name_entry = Entry(View_Accepted_order_frame)
944     food_name_entry.pack()
945
946     food_qty_lbl = Label(View_Accepted_order_frame, text="Food Qty:")
947     food_qty_lbl.pack()
948     food_qty_entry = Entry(View_Accepted_order_frame)
949     food_qty_entry.pack()
950
951     price_lbl = Label(View_Accepted_order_frame, text="Total Price:")
952     price_lbl.pack()
953     price_entry = Entry(View_Accepted_order_frame)
954     price_entry.pack()
955
956     # Employee Details
957     emp_id_lbl = Label(View_Accepted_order_frame, text="Employee ID:")
958     emp_id_lbl.pack()
959     emp_id_entry = Entry(View_Accepted_order_frame)
960     emp_id_entry.pack()
961
962     ename_lbl = Label(View_Accepted_order_frame, text="Assign an Employee")
963     ename_lbl.pack()
964     emp_name_entry = Entry(View_Accepted_order_frame)
965     emp_name_entry.pack()

```

```

966
967 emp_phone_lbl = Label(View_Accepted_order_frame, text="Employee Phone:")
968 emp_phone_lbl.pack()
969 emp_phone_entry = Entry(View_Accepted_order_frame)
970 emp_phone_entry.pack()
971
972
973 def Customer_window():
974     Window.title("CUSTOMER WINDOW - T. NIVEDHA")
975     Customer_window_frame=Toplevel(Window) #define a frame for placing the widgets,but top level object does not have pack() method
976     labelC=Label(Customer_window_frame,text=" CUSTOMER REGISTER / VIEW ",font='arial 20 bold', bg='light green')
977     labelC.pack(pady=30)
978     b1=Button(Customer_window_frame, text="Customer Registration",font="arial 20 bold",bg='cyan',command=lambda: Customer_Register_ShowDetails(1))
979     #store button will be disabled if 0 is passed, enabled if 1 is passed
980     b2=Button(Customer_window_frame, text=" Place Order ",font="arial 20 bold",bg='cyan',command=lambda: Cust_order())
981     b3=Button(Customer_window_frame, text="View Accepted Order",font="arial 20 bold",bg='cyan',command=lambda: CutomerView_AcceptedOrder())
982     b4=Button(Customer_window_frame, text="Exit",font='arial 20 bold',command=Customer_window_frame.destroy,bg='violet')
983
984     # Buttons on the left side, fill vertical space
985     for button in (b1, b2, b3):
986         button.pack(pady=10) # Pack to left, fill vertically with padding
987
988 def Restaurant_window():
989     Window.title("RESTAURANT WINDOW - T. NIVEDHA")
990     Restaurant_window_frame = Toplevel(Window) #define a frame for placing the widgets,but top level object does not have pack() method
991     ###This frame will be displayed at the top of the Window instance. should not use pack()
992     labelR=Label(Restaurant_window_frame,text=" RESTAURANT MANAGER SYSYEM ",font='arial 25 bold', bg='light blue')
993     labelR.pack(pady=20)
994     rest_but1=Button(Restaurant_window_frame,text="Add / View List of Foods Available",font="arial 20 bold",bg='cyan',command=lambda: Add_food_list())
995     rest_but2=Button(Restaurant_window_frame,text=" Register / View Employee Details ",font="arial 20 bold",bg='cyan',command=lambda: Employee_Register_OR_Show())
996     rest_but3=Button(Restaurant_window_frame,text=" Manage Orders ",font="arial 20 bold",bg='cyan',command=lambda: Rest_manageOrder())
997     rest_but4=Button(Restaurant_window_frame,text="View Customer Registration Details",font="arial 20 bold",bg='cyan',command=lambda: Customer_Register_ShowDetails(2))
998     rest_but5=Button(Restaurant_window_frame,text="Exit",font="arial 20 bold",command=Restaurant_window_frame.destroy,bg='violet')
999
1000     # Buttons on the left side, fill vertical space
1001     for button in (rest_but1,rest_but2, rest_but3, rest_but4,rest_but5):
1002         button.pack(pady=10) # Pack to left, fill vertically with padding
1003
1004
1005 def Main_window():
1006     Window.title("MAIN WINDOW - T. NIVEDHA")
1007     #Label for the main window should use only Window instance i.e it will be displayed in main window tilte
1008     Window_label=Label(Window,text=" FOOD ORDERING SYSTEM ",font="arial 40 bold",bg='light blue')
1009     Window_label.pack(pady=40)
1010     #This frame also uses Windows instance i.e. it will be displayed in the Main window
1011     Main_window_frame=Frame(Window) #define a frame for looping
1012
1013     ###b1=Button(Main_window_frame,text="Restaurant",font="arial 20 bold",bg='cyan',command=lambda: Restaurant_window())
1014     ###Frame name should not be given in button class for this menu window because they are to be placed in Window instance
1015     b1=Button(text="Restaurant",font="arial 20 bold",bg='cyan',command=lambda: Restaurant_window())
1016     b2=Button(text="Customer",font="arial 20 bold",bg='cyan',command=lambda: Customer_window())
1017     b3=Button(text="Exit",font='arial 20 bold',command=Window.destroy,bg='violet')
1018
1019     # Buttons on the left side, fill vertical space
1020     for button in (b1, b2, b3):
1021         button.pack(pady=10) # Pack to left, fill vertically with padding
1022
1023     Main_window_frame.mainloop() ###This should not be removed
1024
1025 Main_window()
1026
1027

```

5. CONCLUSION AND FUTURE WORK

The “FOOD ORDERING AND MANAGEMENT SYSTEM WITH SMART USER INTERFACES AND ORDER TRACKING” system was developed and tested successfully. The advantages, limitations and future work for this system are given below.

4.1 Advantages of the System Developed

The **System** developed in this project offers several specific advantages:

1. User-Friendly Interface:

- The use of separate panels for customer and restaurant staff interactions (e.g., registration, order placement, and order management) creates a clear and intuitive interface, making it easy for users to navigate and perform tasks.

2. Automated Customer and Employee ID Generation:

- Unique ID generation for customers and employees reduces the need for manual data entry, minimizes duplicate records, and ensures smooth identification for order and employee tracking.

3. Dynamic View Mode Switching:

- The system’s panels can dynamically switch between registration and viewing modes (e.g., pressing CTRL + ‘v’ to view details), which allows efficient use of the same interface (**Smart Interface**) for multiple tasks and improves user experience.

4. Error Handling and Verification:

- The system incorporates customer ID and phone number verification for order placement, helping to ensure accuracy in customer records and order details. Error messages and prompts provide immediate feedback if issues arise, reducing data entry errors.

5. Efficient Order Management for Staff:

- The restaurant staff can assign specific employees to orders, view accepted orders, and manage food item availability, allowing efficient order handling and ensuring that resources are appropriately allocated.

6. Data Centralization and Accessibility:

- By centralizing customer, employee, food, and order data in a MySQL database, the system enables quick access to relevant information, making it easy for managers to retrieve, update, and analyze data as needed.

7. **Simplified Food Inventory Management:**

- The “Add Food to the List” function allows staff to update available food items, quantities, and prices, ensuring that the menu is always up-to-date and ready for customer orders.

8. **Modularity and Reusability:**

- Functions like Customer_Register_ShowDetails and Rest_manageOrder are modular and reusable, which simplifies maintenance and makes it easier to expand or customize the system for future needs.

These advantages make the system developed in this code a robust, flexible, and efficient tool for managing food ordering and customer service operations within a restaurant.

4.2 Limitations

1. **Basic Security Measure:** While the customer’s phone number is verified for each order and the customer ID is used as a password, additional security measures, such as encryption, could be implemented.
2. **Desktop-Based Interface:** The Tkinter-based interface limits the system to desktop environments. A web-based or mobile app interface could broaden accessibility. But it will require a Web Server for testing.
3. **Dependency on Database Connectivity:** The system relies on a database for storing and retrieving information. Issues with database connectivity could impact system performance.
4. **Dependency on Manual Input:** Restaurant staff need to manually update and manage orders, food items, and employee details, which could introduce errors or delays, particularly during busy periods.

The above limitations will be addressed step by step during further development and studies.

4.3 Future Work

1. **Single-Restaurant Focus:** The current system is designed for a single restaurant. Expanding to a multi-restaurant chain would require additional features like centralized inventory management and franchise-specific settings.

2. **Limited Customer Interaction Features:** Customers cannot customize orders or add special requests, nor can they see a detailed menu with item descriptions or images, which may limit customer engagement.
3. **Limited Payment Integration:** The system may not support a wide range of payment gateways or digital payment options.
4. **Web-Based Interface:** Develop a web-based interface to allow customers to order from any device with internet access.
5. **Mobile App:** Create a mobile app for both customers and restaurant staff to enhance accessibility and convenience.
6. **Cloud Deployment:** Deploy the system on a cloud platform for scalability and reliability.
7. **Lack of Advanced Analytics:** While the system can generate basic reports, it may not provide advanced analytics and insights for strategic decision-making. Adding analytics tools for tracking popular items, peak hours, and customer preferences could assist restaurant managers in decision-making and inventory management.
8. **Integration with Delivery Services:** Integrate with popular delivery services to offer seamless delivery options.
9. **Loyalty Program:** Implement a loyalty program to reward repeat customers and encourage repeat business.
10. **Real-time Inventory Management:** Enhance inventory management with real-time updates and automated reordering.
11. **Security Enhancements:** Strengthen security measures to protect sensitive customer and business data.
12. **Accessibility:** Ensure the system is accessible to users with disabilities by following accessibility guidelines.
13. **User Experience Optimization:** Continuously refine the user interface and user experience based on feedback and usage data.

By addressing these limitations and exploring future opportunities, the system can evolve into a more comprehensive and versatile food ordering and management solution.

6. BIBLIOGRAPHY

- 1 Computer science with Python textbook class 12th by Sumita Arora
- 2 Tkinter Tutorial <https://www.pythontutorial.net/tkinter/>
- 3 W3Schools Online Web Tutorials <https://www.w3schools.com/>